

CSC340 - ASSIGNMENT 4 - WRITTEN ANSWERS

Student Name: Khang Nguyen Jason Anousaya

PART 1 - RECURSIVE FUNCTION

PART 1 - QUESTION 2: RECURSIVE CALL TREE FOR J(6)

```
seriesRecursive(6)
  seriesRecursive(5)
    seriesRecursive(4)
      seriesRecursive(3)
        seriesRecursive(2) = 1
        seriesRecursive(1) = 1
        seriesRecursive(0) = 0
        Result:  $1 + 2*1 + 4*0 = 3$ 
      seriesRecursive(2) = 1
      seriesRecursive(1) = 1
      Result:  $3 + 2*1 + 4*1 = 9$ 
    seriesRecursive(3)
      seriesRecursive(2) = 1
      seriesRecursive(1) = 1
      seriesRecursive(0) = 0
      Result:  $1 + 2*1 + 4*0 = 3$ 
    seriesRecursive(2) = 1
    Result:  $9 + 2*3 + 4*1 = 19$ 
  seriesRecursive(4)
    seriesRecursive(3)
      seriesRecursive(2) = 1
      seriesRecursive(1) = 1
      seriesRecursive(0) = 0
      Result:  $1 + 2*1 + 4*0 = 3$ 
    seriesRecursive(2) = 1
    seriesRecursive(1) = 1
    Result:  $3 + 2*1 + 4*1 = 9$ 
  seriesRecursive(3)
    seriesRecursive(2) = 1
    seriesRecursive(1) = 1
    seriesRecursive(0) = 0
    Result:  $1 + 2*1 + 4*0 = 3$ 
```

Final Result for $\text{seriesRecursive}(6) = 19 + 2*9 + 4*3 = 19 + 18 + 12 = 49$

PART 1 - QUESTION 3: MEMOIZATION

(a) Definition of Memoization:

Memoization is an optimization technique that speeds up recursive algorithms by storing the results of expensive function calls and returning the cached result when the same inputs occur again. It transforms an exponential time algorithm into a linear one by avoiding redundant calculations.

For example, in the seriesRecursive function, calculating $J(6)$ requires calculating $J(5)$, $J(4)$, and $J(3)$. But calculating $J(5)$ also requires calculating $J(4)$ and $J(3)$ again. With memoization, we would store the results of $J(4)$ and $J(3)$ the first time they're calculated and reuse them instead of recalculating.

Source: <https://www.geeksforgeeks.org/what-is-memoization-a-complete-tutorial/>

(b) Strategy to Implement Memoization for seriesRecursive:

To implement memoization for the seriesRecursive function, one would first create a storage structure such as an array or a map to cache previously computed values. The recursive function should be modified to check whether the result for a given input n is already present in the cache. If it is, the function should return the cached result immediately. If not, the function should compute the value recursively, store the result in the cache, and then return it. At the start, base cases such as $J(0) = 0$, $J(1) = 1$, and $J(2) = 1$ should be initialized in the cache. This approach ensures that the function prioritizes cache lookups over redundant recursive calculations. For larger values of n , this strategy significantly reduces the number of recursive calls required. As a result, the time complexity improves from exponential ($O(3^n)$) to linear ($O(n)$), and the space complexity becomes $O(n)$, as only up to n distinct values need to be stored.

PART 2 - SORTING ALGORITHMS

Random Array: [64, 25, 12, 22, 11, 90, 33, 7]

BASIC SORTING ALGORITHM: SELECTION SORT

Initial Array: [64, 25, 12, 22, 11, 90, 33, 7]

Iteration 1:

- Find minimum in positions 0-7: 7 at position 7
- Swap 64 and 7

Array after iteration: [7, 25, 12, 22, 11, 90, 33, 64]

Iteration 2:

- Find minimum in positions 1-7: 11 at position 4
- Swap 25 and 11

Array after iteration: [7, 11, 12, 22, 25, 90, 33, 64]

Iteration 3:

- Find minimum in positions 2-7: 12 at position 2
- Already in correct position, no swap needed

Array after iteration: [7, 11, 12, 22, 25, 90, 33, 64]

Iteration 4:

- Find minimum in positions 3-7: 22 at position 3
- Already in correct position, no swap needed

Array after iteration: [7, 11, 12, 22, 25, 90, 33, 64]

Iteration 5:

- Find minimum in positions 4-7: 25 at position 4
- Already in correct position, no swap needed

Array after iteration: [7, 11, 12, 22, 25, 90, 33, 64]

Iteration 6:

- Find minimum in positions 5-7: 33 at position 6
- Swap 90 and 33

Array after iteration: [7, 11, 12, 22, 25, 33, 90, 64]

Iteration 7:

- Find minimum in positions 6-7: 64 at position 7
- Swap 90 and 64

Array after iteration: [7, 11, 12, 22, 25, 33, 64, 90]

Final Sorted Array: [7, 11, 12, 22, 25, 33, 64, 90]

FAST SORTING ALGORITHM: MERGE SORT

Initial Array: [64, 25, 12, 22, 11, 90, 33, 7]

Divide Step 1:

- Divide into: [64, 25, 12, 22] and [11, 90, 33, 7]

Divide Step 2:

- Divide [64, 25, 12, 22] into: [64, 25] and [12, 22]
- Divide [11, 90, 33, 7] into: [11, 90] and [33, 7]

Divide Step 3:

- Divide [64, 25] into: [64] and [25]
- Divide [12, 22] into: [12] and [22]
- Divide [11, 90] into: [11] and [90]

- Divide [33, 7] into: [33] and [7]

Merge Step 1:

- Merge [64] and [25] into [25, 64]
- Merge [12] and [22] into [12, 22]
- Merge [11] and [90] into [11, 90]
- Merge [33] and [7] into [7, 33]

Merge Step 2:

- Merge [25, 64] and [12, 22] into [12, 22, 25, 64]
- Merge [11, 90] and [7, 33] into [7, 11, 33, 90]

Merge Step 3:

- Merge [12, 22, 25, 64] and [7, 11, 33, 90] into [7, 11, 12, 22, 25, 33, 64, 90]

Final Sorted Array: [7, 11, 12, 22, 25, 33, 64, 90]

PART 3 - RECURSIVE SORTING IN LINKEDBAG

PART 3 - QUESTION 3: MERGE SORT IMPLEMENTATION

(a) How GenAI merge sort code was checked for correctness:

The merge sort algorithm generated by AI was verified for correctness through several validation steps. First, the code was reviewed to ensure it followed the divide-sort-merge structure. It was then tested on a variety of inputs including edge cases (empty and single-node lists), a sorted list, a reverse-sorted list, lists with duplicates, and the assignment-specific list [35, 62, 15, 24, 40, 7]. Each output was checked for correct sorting, preservation of list structure, and completeness. The implementation was analyzed for time and space complexity to ensure it aligned with $O(n \log n)$. Finally, the standalone version was compiled and tested in `mergeSortGenAI.cpp`.

(b) Changes made to adapt code to LinkedException:

Adapting the merge sort code to the LinkedException class involved modifying function signatures to align with LinkedException's structure and adding helper methods such as `mergeSortHelper`, `findMiddle`, and `merge`. Instead of using `ListNode`, the code was adapted for `Node<ItemType>` with corresponding getter and setter methods. Proper memory handling was introduced for dummy nodes to avoid leaks. The code was made generic by using templates and replacing `int` comparisons with type-safe operations. Integration retained the `headPtr` structure of LinkedException and ensured compatibility with other class methods. Error handling for empty and single-element bags was also incorporated.